ROBORESEARCH AI

Robotics AI Research Assistant

A detailed, staged roadmap for building a real, local-first AI chatbot for robotics learning and research.

Project focus
Robotics • Robot Learning • Control Systems • Motion Planning • UAV GNC • Spacecraft GNC

Development environment
Windows 11 · Intel Core i5-1135G7 · 16 GB RAM · Intel integrated graphics

Approach
Free-first · Local-first · Python backend · React frontend · Ollama-powered LLM

Prepared: 3 November 2026 | Version 1.0

How to use this roadmap

This document is the project blueprint. It deliberately separates the minimum viable chatbot from advanced research-assistant features. Follow the phases in order; do not start simulation execution, autonomous agents or fine-tuning before the basic chat and retrieval system is reliable.

Estimated effort: about 16–20 weeks at 8–12 focused hours per week for the planned learning and first robust version. This is a planning estimate, not a guarantee. The first conversational prototype should be possible much earlier.

Project goals

- Build a genuinely conversational chatbot powered by a real language model, not scripted answers.
- Specialize its instructions and knowledge base for robotics, robot learning, control, planning, UAVs and spacecraft GNC.
- Allow users to upload robotics PDFs and ask questions grounded in the uploaded material, with page-aware citations.
- Help users understand equations, papers, algorithms and code while being honest about uncertainty.
- Later connect approved tools for calculations, plotting and simulation experiments.
- Keep the initial development free and local on the user's Windows laptop.

What this project is not

- It is not training a foundation model from scratch. That would require substantial compute and data.
- It is not guaranteed to be an expert or a source of authoritative engineering decisions. Outputs must be verified.
- It is not initially a flight controller or a system that controls real UAVs or spacecraft.
- It is not a paper-writing machine. It can support research workflows, but it must not fabricate citations, results or claims.

Contents

- 1. Product definition and scope
- 2. Recommended technology stack
- 3. System architecture and data flow
- 4. Hardware-aware model strategy
- 5. Detailed development phases and acceptance criteria
- 6. Learning prerequisites
- 7. Project structure and engineering practices
- 8. Evaluation, safety, privacy and security
- 9. Timeline and milestones
- 10. Budget, deployment and future extensions
- 11. Final definition of done and official resources

1. Product definition and scope

Working name: **RoboResearch AI**. The product is a specialized conversational assistant for students and researchers working in robotics and autonomous systems. It combines a language model with a curated knowledge base and later with carefully controlled software tools.

Primary user journeys

User journey	What the application should do	Initial release?
Ask a robotics question	Answer conversationally, explain assumptions, equations and examples.	Yes
Continue a conversation	Retain recent turns and understand follow-up questions.	Yes
Upload a PDF paper	Extract text, index it, answer questions with page/source references.	Phase 2
Compare papers	Retrieve evidence from two or more papers and compare methods and limitations.	Phase 3
Generate code	Draft and explain Python examples for algorithms and simulations.	Phase 4
Run calculations	Use controlled Python tools for numerical calculations and plots.	Phase 5
Run simulations	Launch approved, predefined simulation jobs and show outputs.	Later
Search current literature	Use a web/search integration with source and date tracking.	Later

Knowledge domains

Domain	Topics to support
Robotics foundations	Coordinate frames, transformations, forward/inverse kinematics, rigid-body motion.
Dynamics and control	Newton–Euler, Lagrangian dynamics, PID, state-space, stability, MPC, optimal control.
Planning and autonomy	A*, Dijkstra, RRT/RRT*, trajectory optimization, collision avoidance, navigation.
Robot learning	Supervised learning, imitation learning, reinforcement learning, policy/value methods.
UAV GNC	Quadrotor dynamics, attitude/position control, state estimation, path planning, disturbances.
Spacecraft GNC	Orbital mechanics, attitude dynamics, relative navigation, rendezvous, docking, maneuver planning.
Research methods	Paper reading, experimental design, baselines, metrics, reproducibility, limitations.

Scope discipline

The first release should answer general robotics questions through the local LLM. It should not pretend to have a complete robotics textbook embedded in its weights. The second release adds retrieval from selected documents. Later releases add code tools and simulation. This staged approach keeps the application testable and achievable on 16 GB RAM.

2. Recommended technology stack

Use a small number of well-understood components. Avoid adding a large agent framework before the basic application is working.

Layer	Chosen tool	Why / role
Operating system	Windows 11	Your existing development environment.
Editor	VS Code	Python, TypeScript, Git and terminal integration.
Version control	Git + GitHub	Track changes, document work and maintain a portfolio repository.
Frontend	React + Vite + TypeScript	Chat interface, message state, streaming display and upload UI.
Styling	Tailwind CSS	Consistent responsive interface without a large custom CSS system.
Backend API	Python + FastAPI	Async API endpoints, validation, streaming and integration with Ollama.
Data validation	Pydantic	Validate API requests, settings and structured tool inputs.
Local LLM runtime	Ollama	Run quantized open models locally and expose a local HTTP API.
Initial chat model	Qwen3.5 4B (test first)	Small multimodal-capable model; CPU inference may be slow.
Fallback model	Qwen3.5 2B or 0.8B	Use if 4B causes memory pressure or unacceptable latency.
Embedding model	nomic-embed-text	Create vector representations for semantic retrieval.
PDF extraction	PyMuPDF	Extract PDF text and page-level content for indexing.
Initial vector store	Chroma persistent local store	Simple local semantic search during early RAG development.
Initial chat database	SQLite	Store users (if needed locally), conversations and messages with minimal setup.
Later database option	PostgreSQL + pgvector	Move to relational + vector storage if the application grows.
Testing	pytest + HTTPX; Vitest + React Testing Library	Backend API and frontend behavior tests.
Plots	Matplotlib / Plotly	Show simulation and evaluation results.
Simulation later	Gymnasium + MuJoCo or PyBullet	Build controlled simulation environments and experiments.
Containerization later	Docker	Isolate approved code execution and make deployment reproducible.

Why not start with LangChain or a multi-agent framework?

At first, implement the request flow directly with FastAPI, the Ollama API and a small set of Python modules. This teaches what each component does and reduces hidden complexity. Consider LangChain or LlamaIndex only when retrieval pipelines, tool orchestration or integrations become complicated enough to justify them.

Important stack choices

- Use Chroma + SQLite for the first local version. PostgreSQL + pgvector is a later upgrade, not a requirement for the prototype.
- Use a provider adapter around the LLM so the application can switch from Ollama to another provider without rewriting chat logic.
- Use a local embedding model; do not send private PDFs to a third-party embedding service by default.
- Pin dependency versions in requirements/lock files and record the model tag used for every evaluation.

3. System architecture and data flow

The frontend should never call Ollama directly. It should call the FastAPI backend. The backend handles conversation history, prompt construction, retrieval, model calls, validation and logging.

Component	Responsibilities
React frontend	Chat view, message composer, streaming display, upload state, citations panel, error states.
FastAPI backend	Authentication later, API validation, conversation orchestration, rate limits, file handling.
Conversation service	Load recent messages, apply system instructions, call model, save response.
LLM adapter	Normalize model requests/responses; initially uses Ollama HTTP/API.
Knowledge service	Parse documents, chunk text, create embeddings, retrieve relevant chunks.
Storage layer	SQLite for conversations and metadata; Chroma for vector embeddings and chunk metadata.
Tool service (later)	Allowlisted calculator, plotting, code validation and simulation job interfaces.
Evaluation harness	Run a fixed question set and record answer quality, retrieval and latency metrics.

Basic chat request flow

1. User sends a message in React.
2. React sends the conversation ID, message and optional settings to FastAPI.
3. FastAPI validates the request and loads the recent conversation turns.
4. The conversation service constructs a system prompt and message list.
5. The LLM adapter sends the request to Ollama running on the same computer.
6. FastAPI streams generated text back to the browser.
7. The backend saves the user message and final assistant response.
8. The UI displays the completed answer and any error or status information.

RAG request flow

1. User uploads a PDF; backend checks file type, size and page count.
2. PyMuPDF extracts text page by page. Store source filename, page number and document ID.
3. Text is split into overlapping chunks; each chunk retains its document and page metadata.
4. The embedding model converts chunks into vectors; Chroma stores vectors and metadata.
5. For a question, embed the query and retrieve the most relevant chunks.
6. Pass the question plus selected chunks to the LLM with an instruction to answer only from evidence when the question concerns the uploaded document.
7. Display citations linked to document name and page. If evidence is insufficient, say so.

Architecture principles

- Separate frontend, API, model provider, retrieval and storage into independent modules.
- Keep model-specific code behind an adapter interface.
- Make document ingestion repeatable and idempotent; avoid indexing the same file repeatedly.
- Keep source metadata with every chunk so citations can be reconstructed.
- Do not let a retrieved PDF issue executable instructions. Retrieved content is evidence, not trusted system instructions.

4. Hardware-aware model strategy

Your laptop: 11th Gen Intel Core i5-1135G7, 16 GB RAM, Intel integrated graphics, Windows 11. Plan for CPU-first inference. The integrated graphics should not be assumed to provide useful acceleration for every model/runtime combination.

Model plan

Model / component	Role	Recommendation
Qwen3.5 4B via Ollama	Main local chat model	Try first. The published Ollama tag is around 3.4 GB on disk; runtime memory is higher than the download size.
Qwen3.5 2B	Fallback chat model	Try if 4B is slow or memory constrained.
Qwen3.5 0.8B	Very lightweight fallback	Useful for testing wiring and simple interactions; expect weaker reasoning.
nomic-embed-text	Embeddings	Run separately for document retrieval; embedding workload is usually much lighter than LLM generation.

Practical constraints

- Close memory-heavy applications during local inference, especially browsers with many tabs.
- Start with a 4,096-token context setting; a large context increases memory use and can reduce responsiveness.
- Keep the first RAG retrieval small (for example, 3–5 relevant chunks) and tune it using evaluation rather than guessing.
- Do not load a 9B or larger model as the default on this laptop. A model may technically load but still be too slow or unstable for productive use.
- Record prompt length, generation time, tokens per second if available, RAM usage and model tag.
- If local inference is too slow, keep development local but optionally use a free/paid hosted API through the provider adapter. Never assume free API quotas are permanent.

Recommended initial Ollama commands

```
ollama pull qwen3.5:4b
ollama run qwen3.5:4b
ollama pull nomic-embed-text
ollama list
```

Ollama API endpoint (local only)

```
http://localhost:11434/api/chat
http://localhost:11434/api/embeddings
```

Note: model tags and sizes can change. Confirm the current tag in the Ollama model library before downloading. The 4B tag is listed in the official Ollama library.

5. Detailed development phases

Phase 0 — Project setup and baseline

Goal: create a clean, documented development environment before building features.

- Install VS Code, Git, Python 3.11 or 3.12, Node.js LTS and Ollama.
- Create a Git repository and a README describing the project, hardware and setup steps.
- Create separate frontend and backend folders.
- Create and activate a Python virtual environment; record dependencies.
- Install Ollama and test a small model from PowerShell.
- Measure basic model latency with a fixed set of 5–10 prompts.

Deliverable: reproducible local environment and a baseline performance note.

Acceptance criteria: project runs from documented commands; model responds; Git history contains meaningful commits; no API key or paid service is required.

Phase 1 — A real conversational chatbot

Goal: complete a functional text chatbot with multi-turn conversation and streaming.

Learn	Implement
Python modules, functions, classes, type hints, exceptions and virtual environments.	FastAPI app with /health, /chat and conversation endpoints.
HTTP, JSON, REST, request/response and status codes.	React chat page with message list and composer.
Async basics and streaming concepts.	Ollama adapter that calls the local chat API.
React components, state, props, effects and forms.	Streaming response display, stop/cancel, loading and error states.
SQLite basics and data modeling.	Persist conversations and messages; support new and reopened chats.

Required first-release features

- Send and receive messages; retain the previous turns in the same conversation.
- Start a new conversation and rename/delete local conversations.
- Stream the answer token-by-token or in chunks.
- Show model name, generation state, errors and retry action.
- Preserve line breaks, code blocks and mathematical notation in responses.
- Allow a user to stop a generation.
- Use a robotics-focused system prompt that asks the model to explain assumptions and admit uncertainty.

Deliverable: a working local chatbot that can hold a multi-turn conversation.

Acceptance criteria: follow-up questions use context; refreshing the browser does not erase saved chats; API failures are shown clearly; no secrets are exposed in frontend code.

Phase 2 — Robotics knowledge base and RAG

Goal: answer questions from selected robotics sources with traceable citations.

Learning topics

- Embeddings: what they represent and what semantic similarity means.
- Chunking: page-aware chunks, overlap, headings and metadata.
- Vector search: nearest neighbors, similarity scores and retrieval quality.
- RAG prompting: evidence-grounded answers, source attribution and abstention.
- Evaluation: precision/recall at k, hit rate, answer faithfulness and citation correctness.

Implementation tasks

1. Build a document upload endpoint with file-size, MIME-type and page-count limits.
2. Extract text with PyMuPDF, preserving page numbers and document metadata.
3. Normalize text carefully; do not strip equations or symbols indiscriminately.
4. Chunk by page/section with configurable chunk size and overlap.
5. Generate embeddings using nomic-embed-text through Ollama.
6. Store chunks and vectors in a persistent local Chroma collection.
7. Retrieve top-k chunks for each document question and include source metadata.
8. Display citations such as [Document name, p. 12] and let users inspect the source excerpt.
9. Support document removal and re-indexing.

Initial RAG safeguards

- If no relevant evidence is retrieved, the assistant should say it cannot find the answer in the uploaded documents.
- Distinguish the model's general knowledge from claims supported by the user's documents.
- Do not fabricate page numbers, paper titles, authors or quotations.
- Treat text inside PDFs as untrusted content; ignore instructions in documents that attempt to override the assistant's rules.
- For scanned PDFs, report that text extraction may be incomplete; OCR is a later enhancement.

Deliverable: upload a paper and ask questions with page-aware citations.

Acceptance criteria: citations resolve to the correct file and page; deleting a document removes its searchable chunks; unsupported questions trigger an uncertainty response.

Phase 3 — Research paper workspace

Goal: support structured reading and comparison of robotics papers.

- Extract title, authors, abstract, headings, references and page count when available.
- Provide structured views: problem, research gap, method, assumptions, datasets/simulators, baselines, metrics, results and limitations.
- Support paper-specific Q&A; with citations.
- Compare two papers using a fixed comparison template and cite each claim.
- Create a reading note with user edits and a list of unresolved questions.
- Flag missing information rather than infer unreported experimental details.

Deliverable: a research-paper workspace for reading one paper and comparing two papers.

Acceptance criteria: every paper-specific factual claim has a source location; the comparison separates what each paper reports from the assistant's interpretation.

Phase 4 — Robotics tutor and programming assistant

Goal: make the chatbot useful for learning robotics concepts and writing code examples.

Tutor capabilities

- Explain concepts at beginner, intermediate and advanced levels.
- Derive equations step by step and define symbols and assumptions.
- Use consistent notation and distinguish coordinate frames and units.
- Generate small quizzes and exercises with hints before revealing solutions.
- Create learning paths for kinematics, dynamics, control, planning, RL, UAV and spacecraft GNC.

Programming capabilities

- Explain Python, NumPy, SciPy, Matplotlib and PyTorch snippets.
- Generate small, self-contained examples for PID, state-space systems, A*, RRT* and basic RL environments.
- Explain code line-by-line and state expected inputs/outputs.
- When debugging, request the traceback, environment and minimal reproducible example.
- Never claim code was executed unless it actually ran in the controlled execution service.

Deliverable: robotics tutor modes and code-assistance responses.

Acceptance criteria: generated examples include dependencies and run instructions; answers state assumptions; a human can verify equations and code.

Phase 5 — Safe tools: calculations, plotting and code execution

Goal: allow the assistant to use narrowly scoped tools rather than merely describe actions.

Start with safe, deterministic tools

- Unit conversion and numerical calculations using Python functions with validated inputs.
- Plot a user-provided CSV or a generated small dataset.
- Solve a simple ODE with SciPy and display the state history.
- Calculate and plot a basic PID response for a defined plant.
- Generate a trajectory for a simple 2D point robot and plot obstacles/path.

Code-execution security

- Do not execute arbitrary model-generated Python in the FastAPI process.
- Use an isolated container or separate restricted worker with CPU, RAM, time and file limits.
- Disable network access by default; mount only a temporary working directory.
- Allow only approved libraries and commands.
- Require user confirmation before starting resource-intensive jobs.
- Log the code, parameters, runtime and generated artifacts for reproducibility.

Deliverable: an allowlisted tool interface for calculations and plots. Arbitrary code execution remains disabled until sandboxing is verified.

Phase 6 — Simulation integration

Goal: support simple, reproducible robotics simulation experiments.

Tool	Use	When
Gymnasium	Standard API for RL environments; begin with toy/custom environments.	First simulation stage
PyBullet	Lightweight physics simulation and simple robot scenes.	Optional first 3D simulator
MuJoCo	Robotics and continuous-control physics simulation.	After basics
Matplotlib / Plotly	Plot state, reward, trajectories and control signals.	Throughout
Gazebo + ROS 2	Robotics middleware, sensors and larger simulated systems.	Advanced stage
PX4 SITL + Gazebo	Autopilot software-in-the-loop for UAV experiments.	Advanced UAV extension

Simulation feature progression

1. Generate a simple 2D kinematic robot environment and plot its path.
2. Implement and compare A* and RRT* in a fixed obstacle map.
3. Create a basic Gymnasium navigation environment with documented observations, actions and rewards.
4. Train a small baseline agent only after validating the environment and reward function.
5. Add disturbances/noise and compare policies across fixed random seeds.
6. Only later connect to MuJoCo, Gazebo or PX4 SITL.

Deliverable: a controlled simulation runner that launches approved experiments and returns plots, logs and metrics.

Phase 7 — Evaluation and quality improvement

Goal: measure whether the assistant is actually useful and reliable, not merely impressive in a few demos.

Create a benchmark set

Start with 80–120 questions written and checked by you, then expand. Cover definitions, derivations, code, paper-grounded questions, ambiguous questions, out-of-scope questions and adversarial instructions embedded in documents.

Evaluation area	What to measure	Example check
General answer quality	Technical correctness, completeness, clarity.	Does it correctly distinguish kinematics from dynamics?
Mathematical reliability	Equation correctness, symbols, units and assumptions.	Are the quadrotor equations dimensionally consistent?
RAG retrieval	Recall@k, hit rate, relevance of retrieved chunks.	Did it retrieve the page containing the method?
Citation accuracy	Whether cited page supports the claim.	Does the cited page actually state that result?
Groundedness	Unsupported claims and hallucinations.	Does it invent a paper's findings?
Refusal / abstention	Correctly says evidence is insufficient.	Does it admit when the uploaded paper omits a detail?
Latency	Time to first token and total response time.	Is local inference usable on the laptop?
Stability	Crash rate, memory use, repeatability.	Can it process repeated uploads without failing?

Evaluation workflow

1. Create a versioned question set with expected answer points and source references.
2. Run the same questions against a fixed model tag and settings.
3. Record prompts, retrieved chunks, citations, output, timing and errors.
4. Manually score a representative sample; use automated checks only as support.
5. Classify failures: model knowledge, retrieval miss, bad chunking, prompt issue, citation bug or UI issue.
6. Change one component at a time and rerun the benchmark.

Do not optimize only for longer or more confident answers. Prefer a shorter answer that is accurate, sourced and honest about uncertainty.

Phase 8 — Packaging, deployment and maintenance

- Add configuration through environment variables and a documented .env.example; never commit secrets.
- Add structured logs with sensitive text minimized or redacted.
- Add request size limits, file upload limits, rate limits and safe error messages.
- Add database backup/export and document deletion controls.
- Package the backend and frontend for local use; use Docker only when it helps reproducibility.
- For public deployment, move away from laptop-local Ollama to a properly secured model host or provider API; a local desktop API is not a production hosting plan.
- Document model license, dataset/document rights and third-party service terms.

Deliverable: a documented local application with repeatable setup, tests, backups and a clear deployment plan.

6. Learning prerequisites

You do not need to master every subject before starting. Learn each topic just before the phase that needs it, and implement small exercises as you go.

Skill	What to learn	Why it matters
Python	Functions, classes, modules, typing, exceptions, files, virtual environments, async basics.	Backend, RAG and tools.
HTTP / APIs	Methods, JSON, status codes, headers, streaming, CORS.	Frontend-backend-model communication.
JavaScript / TypeScript	ES modules, async/await, types, React state and effects.	Build the interactive chat UI.
SQL	Tables, primary keys, foreign keys, CRUD, transactions.	Persist conversations and document metadata.
LLM concepts	Tokens, context windows, prompts, sampling, quantization, inference.	Understand model behavior and hardware limits.
Embeddings / retrieval	Vector similarity, chunking, metadata filters, top-k retrieval.	Build document Q&A.;
Robotics foundations	Frames, vectors, matrices, kinematics, dynamics, control, planning.	Design domain prompts and validate answers.
Testing	Unit tests, API tests, integration tests, regression tests.	Prevent breaking features as the project grows.
Security	Input validation, file handling, secrets, prompt injection, sandboxing.	Protect the local app and later users.

Suggested learning resources

Subject	Resource
Python	Python official tutorial: https://docs.python.org/3/tutorial/
FastAPI	https://fastapi.tiangolo.com/tutorial/
React	https://react.dev/learn
Vite	https://vite.dev/guide/
Tailwind CSS	https://tailwindcss.com/docs
Ollama	https://docs.ollama.com/
Qwen3.5 model	https://ollama.com/library/qwen3.5
Sentence Transformers	https://www.sbert.net/
Chroma	https://docs.trychroma.com/
PyMuPDF	https://pymupdf.readthedocs.io/
Gymnasium	https://gymnasium.farama.org/
MuJoCo	https://mujoco.org/
PyBullet	https://pybullet.org/
ROS 2	https://docs.ros.org/
PX4 simulation	https://docs.px4.io/main/en/simulation/

Learning strategy

For each feature, first learn the minimum concepts, then build a small isolated exercise, then integrate it. For example, before implementing RAG, create a small script that extracts text from one PDF, splits it into chunks and searches those chunks. Only then connect it to the chat endpoint.

7. Repository structure and engineering workflow

Suggested repository layout

```
robo-research-ai/ ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ main.py ■ ■ ■■■ api/ ■ ■ ■■■ core/ # config, logging, prompts ■ ■ ■■■ services/ # chat, model, retrieval, documents ■ ■ ■■■ providers/ # Ollama adapter; future API adapters ■ ■ ■■■ db/ # models, repositories, migrations ■ ■ ■■■ tools/ # allowlisted tools only ■ ■ ■■■ schemas/ # Pydantic request/response models ■ ■■■ tests/ ■ ■■■ requirements.txt ■ ■■■ .env.example ■■■ frontend/ ■ ■■■ src/ ■ ■ ■■■ components/ ■ ■ ■■■ pages/ ■ ■ ■■■ hooks/ ■ ■ ■■■ services/ ■ ■ ■■■ types/ ■ ■ ■■■ styles/ ■ ■■■ package.json ■■■ data/ ■ ■■■ uploads/ ■ ■■■ chroma/ ■ ■■■ exports/ ■■■ evaluation/ ■ ■■■ questions/ ■ ■■■ results/ ■ ■■■ scripts/ ■■■ docs/ ■ ■■■ architecture.md ■ ■■■ decisions/ ■ ■■■ user-guide.md ■■■ README.md ■■■ .gitignore
```

Engineering rules

- Use Git branches or small commits for each feature; do not build the entire application in one giant change.
- Write a short architecture decision note when choosing a database, model or retrieval strategy.
- Keep model prompts in versioned files rather than scattering long strings throughout the code.
- Keep configuration separate from source code.
- Write tests for the chat API, persistence, document indexing, retrieval and citations.
- Use type hints and clear names; add docstrings where behavior is not obvious.
- Keep generated data, local model files, secrets and virtual environments out of Git.
- Document exact setup commands so the project can be recreated on another computer.

Suggested Git milestones

Tag	Meaning
v0.1	Local environment and model API test
v0.2	Basic multi-turn chat
v0.3	Streaming and persistent conversation history
v0.4	PDF upload and text extraction
v0.5	RAG retrieval with source citations
v0.6	Paper workspace and comparison
v0.7	Robotics tutor and code helper
v0.8	Safe calculation and plotting tools
v0.9	Simulation job prototype
v1.0	Evaluated, documented local release

8. Safety, privacy and reliability

Because this application will discuss control systems, UAVs and spacecraft, it must be explicit about the limits of generated answers. Treat it as a learning and research aid, not as a flight-qualified engineering system.

Technical reliability

- Ask the assistant to define coordinate frames, units, assumptions and sign conventions when deriving equations.
- For control and dynamics answers, encourage verification against textbooks, papers or simulation.
- Do not let the assistant invent citations. RAG citations must come from retrieved document metadata.
- Label generated code as unverified until tests or execution confirm it.
- Never use chatbot-generated control commands on a real UAV or spacecraft.

Privacy and file handling

- Keep Ollama bound to localhost during development; do not expose its port publicly.
- Validate uploads by file type, size and content; use safe generated filenames and prevent path traversal.
- Store user documents locally and provide delete/export functionality.
- Avoid uploading copyrighted books or papers unless you have the right to use them; personal study use and redistribution are different questions.
- Do not send confidential research or personal data to external model APIs without understanding their data policies.

Prompt injection and tool security

- Treat retrieved documents and user-provided text as untrusted input.
- Separate system instructions from retrieved evidence.
- Never allow document text to trigger tools or override system policy.
- Use an allowlist for tools and validate every argument server-side.
- Require confirmation for expensive or destructive operations.
- Sandbox any future code execution; do not execute generated code inside the main backend process.

Model and data licensing

Before distributing or deploying the application, check the license and usage terms for the selected model, embedding model, datasets, papers and software dependencies. A model being downloadable for free does not automatically mean every commercial or redistribution use is unrestricted.

Minimum privacy posture for the first local version

Control	Initial requirement
Network exposure	Localhost only; no public API ports.
Secrets	No API keys needed for local-only mode; use environment variables if later added.
Uploads	Local storage, file limits, explicit delete.
Logs	Avoid logging full documents and sensitive user content by default.
Code execution	Disabled until an isolated sandbox is implemented and tested.
Simulation	Predefined, bounded jobs only; no real vehicle control.

9. Timeline and milestones

The schedule below assumes approximately 8–12 focused hours each week. It is intentionally conservative because you are learning some of the stack while building it.

Weeks	Focus	Milestone / evidence
1	Environment, Python refresh, Git, Ollama test	Documented setup; baseline model runs.
2–3	FastAPI, HTTP, React basics, simple LLM call	First local chat round-trip.
4–5	Streaming, conversation state, SQLite	Multi-turn chat with saved history.
6–7	PDF extraction, chunking, embeddings, Chroma	Upload and index a robotics PDF.
8–9	Retrieval, citations, grounded prompting	Document Q&A; with page citations.
10–11	Paper workspace, structured paper analysis	Paper summary and two-paper comparison.
12–13	Robotics tutor, code assistant, tests	Topic-aware tutor and code explanations.
14–15	Safe tools, calculation, plotting	Approved numerical tool calls and plots.
16–18	Gymnasium environment and simulation prototype	Simple simulated robot task with metrics.
19–20	Evaluation, security review, documentation	Version 1.0 local release.

Weekly work rhythm

- 2–3 hours: learn the concepts needed for the current phase.
- 4–6 hours: implement and test the feature.
- 1–2 hours: debug, document and commit code.
- 1 hour: evaluate what works, list failures and plan the next iteration.

If time is limited

Prioritize v0.1–v0.5: a reliable local chat app with RAG and citations. That is already a substantial project. Defer simulation, agents, deployment and advanced paper comparison rather than rushing them.

Milestone review questions

- Can I explain the architecture without reading the code?
- Can I reproduce the setup from the README?
- Do I have tests proving the feature works?
- Can I identify the feature's known limitations?
- Have I measured performance on my actual laptop?

10. Budget and deployment plan

Local-first development budget

Item	Expected initial cost	Notes
Python, FastAPI, React, Vite, Tailwind	NPR 0	Open-source tools.
Ollama and local model	NPR 0	No per-token API fee; consumes disk, RAM, CPU and electricity.
Embedding model and Chroma	NPR 0	Local storage and inference.
SQLite and PyMuPDF	NPR 0	Local database and PDF extraction.
Git and basic GitHub use	NPR 0	Free tier is sufficient for a public code portfolio.
Domain / public hosting	Optional	Not required for local development.
Paid model API	Optional	Only if local inference is too slow or a feature requires a hosted model.
GPU upgrade	Not recommended now	Do not buy hardware before measuring actual bottlenecks.

What 'free' means here

The software and local inference can be used without a usage-based model bill. However, the laptop, electricity, internet connection, storage and your time still have costs. Public deployment may introduce hosting, domain, storage or API expenses.

Local development versus public deployment

Local development	Public deployment
Ollama runs on your laptop.	A server or hosted inference provider runs the model.
SQLite and Chroma can be local folders.	Use managed or server-hosted database/vector storage.
No public user authentication required for a single-user local app.	Authentication, authorization, HTTPS, rate limits and monitoring are required.
Local files stay on the computer.	Define retention, privacy, backups and user deletion policies.
Resource use is limited by laptop CPU/RAM.	Set server capacity, concurrency and cost controls.

Do not deploy the development server as-is. A public release needs a separate security, privacy, performance and licensing review.

11. Final definition of done

The project can be called a successful first release when the following are true:

- A user can start, continue, rename and delete conversations.
- The assistant streams responses from a real local model.
- Conversation history persists between app restarts.
- A user can upload a permitted robotics PDF and ask questions about it.
- Document-grounded answers cite the correct document and page.
- The assistant admits when the document does not contain enough evidence.
- The model provider and embedding provider are isolated behind clean interfaces.
- Automated tests cover core API, storage, retrieval and citation behavior.
- A benchmark report records answer quality, retrieval quality, latency and known limitations.

- The README explains installation, configuration, model download, running, testing and troubleshooting.
- No arbitrary code execution or real-vehicle control is enabled.

What makes this a strong portfolio project?

A strong version is not defined by the number of features. It is defined by sound architecture, reproducible setup, a useful domain-specific retrieval system, transparent evaluation, clear limitations and code you can explain. Those are the qualities that make this a credible engineering project and a useful foundation for later robotics research.

12. Official documentation and references

Use official documentation as the source of truth for installation commands, model tags, API behavior and version changes. The links below were checked while preparing this roadmap; always verify current instructions before installing.

Technology	Official resource
Ollama Windows download	https://ollama.com/download/windows
Ollama documentation	https://docs.ollama.com/
Qwen3.5 model in Ollama	https://ollama.com/library/qwen3.5
Qwen3.5 4B tag	https://ollama.com/library/qwen3.5:4b
Ollama API	https://docs.ollama.com/api
FastAPI tutorial	https://fastapi.tiangolo.com/tutorial/
React learning guide	https://react.dev/learn
Vite guide	https://vite.dev/guide/
Tailwind CSS documentation	https://tailwindcss.com/docs
PyMuPDF documentation	https://pymupdf.readthedocs.io/
Chroma documentation	https://docs.trychroma.com/
Sentence Transformers	https://www.sbert.net/
pgvector	https://github.com/pgvector/pgvector
Gymnasium	https://gymnasium.farama.org/
MuJoCo	https://mujoco.org/
PyBullet	https://pybullet.org/
ROS 2 documentation	https://docs.ros.org/
PX4 simulation	https://docs.px4.io/main/en/simulation/

Final recommendation

Build in this order: local model test → FastAPI chat API → React chat UI → conversation persistence → PDF ingestion → RAG with citations → robotics tutor → safe tools → simulation integration → evaluation and release.

Do not start with fine-tuning, multi-agent orchestration, a large cloud deployment or a full PX4/Gazebo stack. Those are later options. First build a reliable, explainable application that works on the laptop you already own.